Guía rápida de comandos Git

Comandos más usados para trabajar con Git y GitHub: ramas, commits, repositorios remotos y cómo regresar a versiones anteriores

Git vs GitHub: Git es la herramienta que usas en tu terminal para controlar versiones. GitHub es el sitio donde guardas tus repositorios en la nube. Todos estos comandos son de Git y funcionan igual con GitHub, GitLab o Bitbucket. Lo que está entre < > se reemplaza por tu valor (sin los símbolos).

1. Configuración inicial

Se hace una sola vez por computadora.

Comando	Qué hace
`git config --global user.name "Tu Nombre"`	Define tu nombre para los commits.
`git config --global user.email "tu@correo.com"`	Define tu correo (usa el mismo de GitHub).
`git config --global init.defaultBranch main`	Hace que los repos nuevos empiecen con la rama **main**.
`git config --list`	Muestra toda tu configuración actual.
`git --version`	Muestra la versión de Git instalada.

2. Crear o clonar un repositorio

Comando	Qué hace
`git init`	Convierte la carpeta actual en un repositorio Git.
`git clone <url>`	Descarga un repositorio de GitHub a tu computadora.
`git clone <url> <carpeta>`	Clona el repositorio en una carpeta con otro nombre.
`git remote add origin <url>`	Conecta tu repo local con uno de GitHub.
`git remote -v`	Muestra a qué repositorios remotos está conectado.
`git remote set-url origin <url>`	Cambia la URL del remoto (p. ej. si renombraste el repo).

3. Día a día: guardar cambios

El ciclo normal: modificas, agregas (add), confirmas (commit) y subes (push).

Comando	Qué hace
`git status`	Muestra qué archivos cambiaron y cuáles están listos para commit.
`git add <archivo>`	Agrega un archivo al área de preparación (staging).
`git add .`	Agrega todos los cambios de la carpeta actual.
`git commit -m "mensaje"`	Guarda una versión (commit) con un mensaje descriptivo.

Comando	Qué hace
`git commit -am "mensaje"`	Agrega y confirma archivos ya rastreados en un solo paso (no incluye archivos nuevos).
`git commit --amend`	Corrige el último commit (mensaje o archivos olvidados). Úsalo solo si aún no hiciste push.
`git diff`	Muestra los cambios que aún no agregaste con add.
`git diff --staged`	Muestra los cambios que ya agregaste y van a ir en el commit.
`git rm <archivo>`	Borra un archivo y registra el borrado.
`git mv <viejo> <nuevo>`	Renombra o mueve un archivo.

4. Ramas (branches)

Una rama te permite trabajar en algo nuevo sin afectar la rama principal.

Comando	Qué hace
`git branch`	Lista las ramas locales (la actual tiene un *).
`git branch -a`	Lista ramas locales y remotas.
`git branch <nombre>`	Crea una rama nueva (pero no te cambia a ella).
`git switch <nombre>`	Te cambia a otra rama (forma moderna).
`git checkout <nombre>`	Te cambia a otra rama (forma clásica, hace lo mismo).
`git switch -c <nombre>`	Crea una rama nueva y te cambia a ella.
`git checkout -b <nombre>`	Igual que el anterior, forma clásica.
`git switch -`	Regresa a la rama en la que estabas antes.
`git branch -m <nuevo-nombre>`	Renombra la rama actual.
`git branch -d <nombre>`	Borra una rama local (solo si ya fue fusionada).
`git branch -D <nombre>`	Fuerza el borrado de una rama local aunque no esté fusionada.
`git push origin --delete <nombre>`	Borra una rama en GitHub.
`git switch -c <nombre> origin/<nombre>`	Crea una rama local a partir de una rama remota.

5. Trabajar con GitHub (remoto)

Comando	Qué hace
`git push`	Sube tus commits a GitHub.
`git push -u origin <rama>`	Sube una rama nueva por primera vez y la vincula; después basta con git push.
`git pull`	Descarga y fusiona los cambios del remoto en tu rama actual.
`git pull --rebase`	Descarga cambios y coloca tus commits encima (historial más limpio).
`git fetch`	Descarga los cambios del remoto pero NO los aplica todavía.
`git fetch --prune`	Actualiza y limpia referencias a ramas remotas que ya se borraron.

6. Unir ramas: merge y rebase

Primero cámbiate a la rama que va a **recibir** los cambios (normalmente main).

Comando	Qué hace
`git merge <rama>`	Une los cambios de <rama> en la rama actual.
`git merge --no-ff <rama>`	Une creando siempre un commit de merge (deja claro en el historial que hubo una rama).
`git merge --abort`	Cancela un merge que tiene conflictos y deja todo como estaba.
`git rebase <rama>`	Reaplica tus commits encima de <rama>.
`git rebase --continue`	Continúa el rebase después de resolver un conflicto.
`git rebase --abort`	Cancela el rebase.
`git cherry-pick <hash>`	Copia un commit específico de otra rama a la actual.

Resolver conflictos: Git marca las partes en conflicto en el archivo con <<<<<<<, ======= y >>>>>>>. Edita el archivo, deja la versión correcta, borra las marcas y luego haz **git add <archivo>** y **git commit** (o **git rebase --continue** si estabas en un rebase).

7. Ver el historial

Comando	Qué hace
`git log`	Muestra el historial completo de commits.
`git log --oneline`	Historial resumido, un commit por línea (útil para copiar el hash).
`git log --oneline --graph --all`	Historial con dibujo de las ramas.
`git log -p <archivo>`	Historial de cambios de un archivo específico.
`git show <hash>`	Muestra qué cambió en un commit.
`git blame <archivo>`	Muestra quién modificó cada línea y en qué commit.
`git reflog`	Historial de TODO lo que hiciste (incluso commits “perdidos”). Tu salvavidas.

8. Regresar a una versión anterior

El **hash** es el código del commit que ves en **git log --oneline** (ej. a1b2c3d).

Comando	Qué hace
`git restore <archivo>`	Descarta los cambios no guardados de un archivo (vuelve al último commit).
`git restore .`	Descarta TODOS los cambios no guardados.
`git restore --staged <archivo>`	Saca un archivo del staging (deshace el git add) sin perder cambios.
`git restore --source <hash> <archivo>`	Recupera un archivo tal como estaba en un commit anterior.
`git checkout <hash>`	Viaja a un commit para verlo (modo “detached HEAD”). Regresa con git switch main.
`git revert <hash>`	Crea un commit nuevo que deshace ese commit. **Seguro** para ramas ya subidas a GitHub.
`git reset --soft HEAD~1`	Deshace el último commit pero conserva los cambios listos (en staging).
`git reset HEAD~1`	Deshace el último commit y deja los cambios en tus archivos (sin staging).
`git reset --hard HEAD~1`	Deshace el último commit y BORRA sus cambios.
`git reset --hard <hash>`	Regresa todo el proyecto exactamente a ese commit, borrando lo posterior.
`git reset --hard origin/main`	Deja tu rama idéntica a la de GitHub (descarta todo lo local).
`git clean -fd`	Borra archivos y carpetas nuevos que Git no está rastreando.

Cuidado: reset --hard y **clean** borran trabajo. Si ya hiciste push, prefiere **git revert**; un reset sobre commits ya subidos obliga a usar **git push --force** y puede romperle el trabajo a tu equipo. Si borraste algo por error, busca el commit en **git reflog** y regresa con **git reset --hard <hash>**.

9. Guardar cambios temporalmente (stash)

Útil cuando necesitas cambiar de rama pero no quieres hacer commit todavía.

Comando	Qué hace
`git stash`	Guarda tus cambios en un “cajón” y deja la carpeta limpia.
`git stash push -m "descripción"`	Guarda los cambios con un nombre.
`git stash -u`	Incluye también archivos nuevos (no rastreados).
`git stash list`	Lista lo que tienes guardado.
`git stash pop`	Recupera el último stash y lo borra del cajón.
`git stash apply stash@{1}`	Aplica un stash específico sin borrarlo.
`git stash drop stash@{0}`	Borra un stash.

10. Etiquetas (versiones / releases)

Comando	Qué hace
`git tag`	Lista las etiquetas.
`git tag -a v1.0 -m "Versión 1.0"`	Crea una etiqueta en el commit actual.
`git push origin v1.0`	Sube una etiqueta a GitHub.
`git push --tags`	Sube todas las etiquetas.
`git tag -d v1.0`	Borra una etiqueta local.

11. .gitignore

Archivo de texto en la raíz del proyecto con lo que Git debe ignorar (una regla por línea), por ejemplo:

Comando	Qué hace

`node_modules/`
`__pycache__/`
`.env`
`*.log`
`build/`
	Ignora dependencias, caché de Python, variables secretas, logs y carpetas de compilación.

`git rm --cached <archivo>`	Deja de rastrear un archivo que ya habías subido (sin borrarlo de tu PC). Después agrégalo al .gitignore.

12. Flujos típicos

Recetas completas para las situaciones más comunes.

Comando	Qué hace

`git init`
`git add .`
`git commit -m "Primer commit"`
`git branch -M main`
`git remote add origin <url>`
`git push -u origin main`
	Subir un proyecto nuevo a GitHub. Antes crea el repositorio vacío en github.com y copia su URL.

`git switch main`
`git pull`
`git switch -c feature/login`
`# ...trabajas y haces commits...`
`git push -u origin feature/login`
	Trabajar una función nueva en su propia rama. Después abres un Pull Request en GitHub.

`git switch main`
`git pull`
`git merge feature/login`
`git push`
`git branch -d feature/login`
	Fusionar tu rama en main localmente y borrarla al terminar.

`git switch feature/login`
`git fetch`
`git rebase origin/main`
	Actualizar tu rama con lo último de main antes de terminar.

`git stash`
`git switch otra-rama`
`# ...`
`git switch -`
`git stash pop`
	Cambiar de rama sin perder cambios que aún no quieres confirmar.

`git reset --soft HEAD~1`
`git switch -c rama-correcta`
`git commit -m "mensaje"`
	Hice commit en la rama equivocada (sin push): mueve el commit a otra rama.

`git log --oneline`
`git revert <hash>`
`git push`
	Deshacer un commit que ya está en GitHub de forma segura.

Consejos: haz commits pequeños y con mensajes claros ("Agrega login con Google", no "cambios"); haz **git pull** antes de empezar a trabajar; nunca subas contraseñas ni archivos **.env**; y ante la duda, corre **git status** — casi siempre te dice qué hacer.